

ARENDO

Протокол интеграции постамата

Backend <-> WebSocket <-> Android

Готовый документ для backend-разработчика

MVP: команда открытия, состояние двери, подтверждение закрытия и контроль связи

Назначение документа

Документ фиксирует фактический контракт между сервером и контроллером постамата. Его задача - чтобы backend отправлял команды в одном формате, Android однозначно понимал ячейку, а сервер принимал только подтвержденные физические состояния.

Коротко о готовности

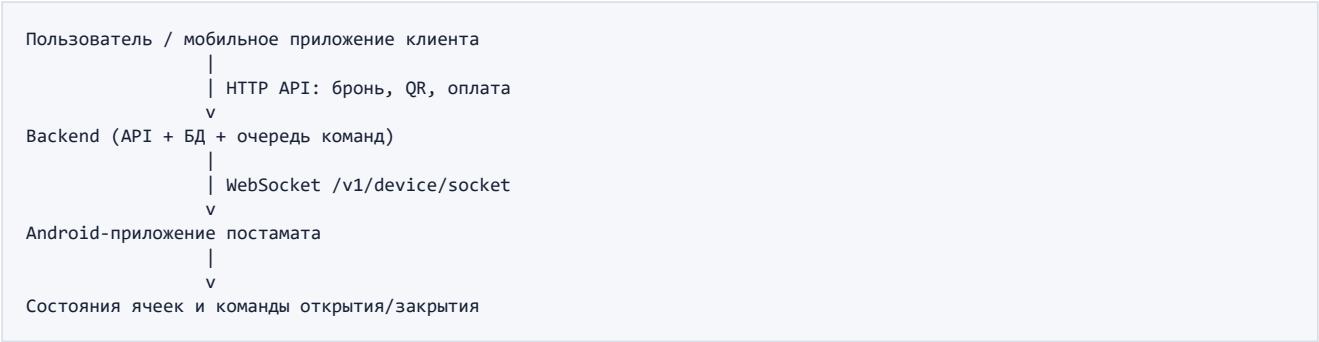
- Android-приложение подключается к WebSocket и отправляет hello/heartbeat.
- Команды open_cell, confirm_closed и cell_report принимаются.
- Ячейки D1-D4 передаются через единый идентификатор cellCode.
- События door_opened и door_closed видны в журнале и отправляются на backend.
- Локальный сервер ноутбука используется для стендовой проверки; production backend должен заменить его своим endpoint.

Что backend должен сделать по этому документу

Реализовать WebSocket endpoint, регистрацию устройства, маршрутизацию команд по postamatId и cellCode, хранение последнего состояния, heartbeat timeout, идемпотентность команд и корректную бизнес-логику аренды. Дизайн мобильного приложения в этот контракт не входит.

1. Архитектура и зоны ответственности

Основной канал - WebSocket. MQTT в текущей архитектуре не используется. Детали аппаратного подключения устройства в этот документ не входят.



Каналы

Канал	Назначение	Контракт
Backend -> Android	Команды открытия, закрытия, отчет и сервис	WebSocket JSON
Android -> Backend	hello, heartbeat, ack и физические события	WebSocket JSON
Реклама	Получение плейлиста и файлов	HTTP; WebSocket только сообщает playlist_changed

Идентификаторы

Поле	Правило
postamatId	Уникальный ID постамата, например tap-7. Не менять между подключениями.
cellCode	Строка с двумя цифрами: 01, 02, 03, 04. В интерфейсе можно показывать D1, D2, D3, D4.
command id	Уникальный ID команды. По нему backend и устройство обеспечивают идемпотентность.
door	Только physical-состояние: open, closed или unknown.
rentalStatus	Бизнес-состояние ячейки. Его считает backend на основе операций аренды и физических событий.

2. Подключение WebSocket и авторизация

Устройство устанавливает одно постоянное соединение с endpoint сервера. Токен передается в HTTP-заголовке WebSocket handshake:

```
GET /v1/device/socket HTTP/1.1
Host: backend.example.com
Upgrade: websocket
Connection: Upgrade
X-Device-Token: <DEVICE_TOKEN>
```

Режим	Пример URL	Правило
Production	wss://backend.example.com/v1/device/socket	Только TLS с проверкой сертификата.
Локальный стенд	ws://:8765/v1/device/socket	Только локальная сеть; не использовать в production.
Тестовый сервер	wss:///v1/device/socket	Требует зарегистрированный и активный device token.

Токен

В production backend хранит только SHA-256 токена и признак active/disabled. Сам токен не печатается в логах и не возвращается в hello/welcome. Неизвестный или отключенный токен должен получить HTTP 401 до установления WebSocket-сессии.

Важно для текущего MVP: локальный сервер ноутбука принимает local-dev для стенда. Это не является production-аутентификацией.

После успешного handshake

Android сразу отправляет hello. Backend отвечает welcome. Только после welcome устройство считается согласованным с конкретным postamatId и получает дальнейшие команды.

Настройки heartbeat

Параметр	Значение	Кто контролирует
heartbeatSec	15 секунд	Android отправляет heartbeat
pingSec	20 секунд	Backend отправляет WebSocket ping
pongWaitSec	45 секунд	Backend закрывает зависшее соединение

3. Сообщения WebSocket

Все сообщения - UTF-8 JSON с обязательным полем type. События и heartbeat могут прийти повторно. Backend должен принимать повтор безопасно: состояние перезаписывается, команды дедуплицируются по id.

3.1 Android -> Backend: hello

```
{
  "type": "hello",
  "postamatId": "map-7",
  "appVersion": "0.2.1",
  "keyVersion": 0,
  "capabilities": ["locks", "door_sensors", "service_open"],
  "cells": [
    {"code": "01", "door": "closed", "lock": "unknown"},
    {"code": "02", "door": "closed", "lock": "unknown"},
    {"code": "03", "door": "closed", "lock": "unknown"},
    {"code": "04", "door": "closed", "lock": "unknown"}
  ],
  "net": {"kind": "wifi"}
}
```

Backend проверяет postamatId, регистрирует online-сессию, сохраняет appVersion/capabilities и отправляет welcome.

3.2 Backend -> Android: welcome

```
{
  "type": "welcome",
  "postamatId": "map-7",
  "postamatName": "Постамат map-7",
  "config": {"heartbeatSec": 15, "pingSec": 20, "pongWaitSec": 45},
  "cells": [
    {"code": "01", "door": "closed", "lock": "unknown", "rentalStatus": "available"},
    {"code": "02", "door": "closed", "lock": "unknown", "rentalStatus": "available"}
  ]
}
```

Массив cells в welcome может содержать все ячейки постамата. rentalStatus - бизнес-расширение backend; физическое door-состояние приходит от устройства.

4. Состояния ячеек и события

Backend получает только внешнее состояние ячейки: D1 open/closed, D2 open/closed, D3 open/closed и D4 open/closed. Способ, которым Android получает это состояние внутри устройства, не является частью backend-контракта.

4.1 heartbeat

```
{
  "type": "heartbeat",
  "cells": [
    {"code": "01", "door": "open", "lock": "unknown"},
    {"code": "02", "door": "closed", "lock": "unknown"}
  ],
  "net": {"kind": "wifi"},
  "problems": []
}
```

Heartbeat - периодический снимок. Он не должен сам запускать оплату или аренду. Backend использует его для online/offline и диагностики.

4.2 event

```
{
  "type": "event",
  "kind": "door_opened",
  "cellCode": "01",
  "detail": {"source": "lock_relay"}
}

{
  "type": "event",
  "kind": "door_closed",
  "cellCode": "01",
  "detail": {"source": "door_sensor"}
}
```

kind	Когда отправляется	Действие backend
door_opened	Android подтвердил открытие ячейки	Зафиксировать открытие; аренду начинать только по успешному ask команды.
door_closed	Android подтвердил закрытие ячейки	Обновить physical state; закрытие аренды подтверждать только в нужном бизнес-контексте.
door_jammed	Есть команда, но механизм не сработал	Создать incident и не считать операцию успешной.
net_degraded	Проблема сети	Показать offline/degraded, не терять последнюю команду и состояние.
power_lost	Питание контроллера потеряно	Зафиксировать incident; после восстановления ждать hello/heartbeat.
tamper	Тревога вскрытия, если появится датчик	Создать incident высокой важности.

Если Android не может достоверно определить lock, передается lock=unknown. Backend не должен заменять unknown на ok самостоятельно.

5. Команды backend -> Android

Каждая команда имеет id, kind, payload, expiresAt и signature. Для MVP signature может быть проверена в production-слое позже, но backend должен уже формировать это поле.

```
{
  "type": "command",
  "id": "cmd-20260919-00001",
  "kind": "open_cell",
  "payload": {"cellCode": "01"},
  "expiresAt": "2026-09-19T12:05:00Z",
  "signature": "<HMAC-SHA256>"
}
```

kind	payload	MVP-результат
open_cell	cellCode: 01..04	Открыть указанную ячейку; вернуть ack result.kind=door_open_ack и отправить door_opened.
confirm_closed	cellCode: 01..04	Проверить, что дверь закрыта; вернуть ack result.kind=door_close_ack или door_not_closed.
cell_report	нет	Вернуть ack с полным cells-снимком вне очереди.
reboot	reason	Зарезервировано; не выполнять при открытой двери.
update_app	version, url, checksum	Зарезервировано для обновления приложения.
rotate_token	token	Зарезервировано для безопасной смены токена.
sync_keys	keys, keyVersion	Зарезервировано для офлайн QR/Ed25519.

Проверки перед выполнением команды

- Проверить, что cellCode существует и относится к этому postamatId.
- Проверить expiresAt. Просроченная команда не должна открывать дверь.
- Не повторять физическое действие при повторном id команды.
- При потере связи команда, которую Android не получил, не должна открыться позже после восстановления.
- Для open_cell backend считает операцию успешной только после ack с door_open_ack.

6. Подтверждения и ошибки

6.1 Успешное открытие

```
{
  "type": "ack",
  "commandId": "cmd-20260919-00001",
  "ok": true,
  "result": {"kind": "door_open_ack", "cellCode": "01"}
}
```

После ack Android отправляет физическое событие door_opened. Backend должен связать ack и событие по commandId/cellCode или по текущей операции.

6.2 Успешное закрытие

```
{
  "type": "ack",
  "commandId": "cmd-20260919-00002",
  "ok": true,
  "result": {"kind": "door_close_ack", "cellCode": "01"}
}
```

6.3 Типовые ошибки

error	Причина	Что делает backend
invalid_cell	Неизвестная ячейка	Не менять состояние аренды; вернуть ошибку API.
expired	Команда просрочена	Закрыть попытку; не открывать дверь повторно.
device_unavailable	Android не смог выполнить физическую команду	Создать incident locker_no_ack, не списывать оплату.
door_not_closed	Android не подтвердил закрытие	Не завершать возврат; показать дверь как open/unknown.
unsupported_command	Команда ещё не реализована на Android	Сохранить ошибку и не повторять бесконечно.

Принцип денег: одна телеметрия door_opened или door_closed сама по себе не двигает деньги. Финансовое состояние меняется только по подтвержденному сценарию backend.

7. Что должен реализовать backend-разработчик

Приоритет	Действие	Критерий готовности
P0	Зарегистрировать postamatId map-7 и активный device token	WebSocket handshake не возвращает 401 для тестового устройства.
P0	Реализовать /v1/device/socket и auth через X-Device-Token	Android получает welcome после hello.
P0	Направить команду open_cell на нужный postamatId/cellCode	Команда D1 не может открыть D2.
P0	Обработать ack door_open_ack и door_close_ack	Аренда/возврат меняются только после правильного ack.
P0	Сохранять cells и события	В админке видны D1 open/closed и время последнего heartbeat.
P0	Heartbeat timeout	Нет heartbeat более 45 секунд -> offline/incident.
P0	Идемпотентность по command id	Повтор сообщения не выполняет физическое действие второй раз.
P1	Команда cell_report	Backend может запросить полный снимок ячеек.
P1	Корреляция операций аренды	Одна бронь связана с postamatId, cellCode, commandId.
P1	Audit log без токенов	Токен никогда не попадает в логи и UI.
P2	rotate_token, update_app, sync_keys	Добавить после стабилизации основного MVP.

Серверный API аренды

Когда клиент завершил оплату и бронь подтверждена, backend выбирает нужный postamatId и cellCode, отправляет open_cell, переводит операцию в pending_open и ждет ack. Только после door_open_ack операция переводится в opened/active. Если ack не пришел или пришел с ошибкой - аренда не стартует, деньги не списываются, создается locker_no_ack.

8. Основной сценарий MVP

1. Бронь и успешная оплата на backend.
2. Backend определяет postamatId=map-7 и cellCode=01.
3. Backend -> Android: command open_cell, cellCode=01.
4. Android выполняет открытие и отвечает ack door_open_ack.
5. Android отправляет event door_opened, cellCode=01.
6. Backend переводит аренду в active/opened.
7. Пользователь закрывает дверь; Android фиксирует закрытие.
8. Android отправляет event door_closed, cellCode=01.
9. Backend отправляет confirm_closed или принимает подтверждение по текущей операции.
10. Android отвечает door_close_ack либо door_not_closed.
11. Backend завершает возврат только при подтвержденном закрытии.

Сценарий потери связи

Ситуация	Ожидаемое поведение
Команда не доставлена	После восстановления Android не выполняет старую просроченную команду.
Команда доставлена, ack потерян	Backend не создает вторую команду без проверки command id и текущего состояния.
Связь восстановилась	Android отправляет hello и heartbeat; backend повторно синхронизирует состояние.
Молчание > 45 сек	postamat offline; активная аренда получает incident.
Дверь реально открыта, ack потерян	Событие door_opened после восстановления помогает диагностике; бизнес-решение принимает backend.

Критично: нельзя отправлять старую open_cell после восстановления связи без проверки expiresAt и idempotency. Иначе дверь может открыться неожиданно.

9. Чек-лист совместного тестирования

Тест	Действие	Ожидаемый результат
T1 Handshake	Подключить Android с активным токеном	hello -> welcome, устройство online
T2 Wrong token	Подключить с неверным токеном	HTTP 401, WebSocket не создается
T3 Open D1	Отправить open_cell/01	ack door_open_ack, D1 open
T4 Close D1	Закрыть D1 штатным способом	event door_closed, D1 closed
T5 Open D4	Отправить open_cell/04	Меняется только D4
T6 Invalid D5	Отправить cellCode 05	ack invalid_cell, физического действия нет
T7 Device unavailable	Сымитировать недоступность устройства	ошибка выполнения, аренда не стартует
T8 Duplicate	Повторить тот же command id	Второго физического действия нет
T9 Expired	Отправить просроченную команду	ack expired, дверь не открывается
T10 Network loss	Отключить Wi-Fi на 60 сек	offline/incident, после возврата hello и heartbeat
T11 Close failure	Оставить дверь открытой	door_not_closed, возврат не завершен
T12 Report	Отправить cell_report	Полный снимок D1-D4 возвращен в ack

Для демонстрации завтра достаточно T1, T3, T4, T5, T7 и T10. Остальные тесты обязательны перед production.

10. Текущий статус и границы MVP

Готово с нашей стороны

- Android-приложение с WebSocket-клиентом постамата.
- Единая модель ячеек D1-D4 и статусов open/closed.
- WebSocket-клиент с hello, welcome, heartbeat, command, ack и event.
- Статусы двери open/closed и отдельный журнал обратной связи.
- Локальный WebSocket-сервер ноутбука и панель проверки команд.
- Проверка работы Android и backend через Wi-Fi.

Не является готовым production

- Локальный сервер ноутбука - только стенд, он не заменяет основной backend.
- В текущем стенде lock=unknown, пока нет отдельного датчика замка.
- Офлайн QR/open, Ed25519, rotate_token, update_app и mTLS требуют отдельной реализации.
- Тестовый TLS без проверки сертификата нельзя оставлять в production.
- Supabase и MQTT в текущем MVP не участвуют; основной канал - WebSocket.

Финальное правило для согласования

Backend управляет бизнес-сценарием, оплатой, бронью и идентификаторами. Android выполняет команды и сообщает только то, что реально видит: D1 open/closed, D2 open/closed и так далее. Поэтому сервер не должен рисовать или считать состояние, которого нет в hello, heartbeat или event.

Контактная точка для backend

При изменении endpoint, postamatId, токена, числа ячеек или формата команд backend-разработчик должен сначала согласовать изменение контракта. В противном случае Android будет подключен, но команда может не дойти до нужной ячейки.